PAGE No.
DATE / /

Module 1 : Introduction to Data Structures.

1.1. Introduction to Data Structures, Concept of ADT
1.2. Types of Data Structures - Linear and Non-linear, Operations on Data structures.

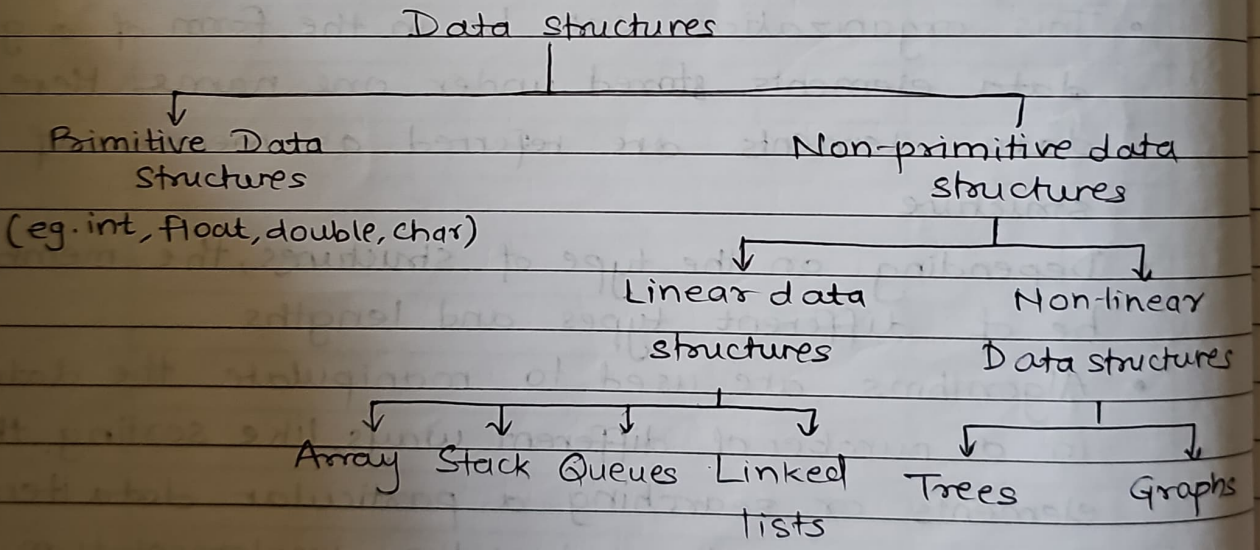
* Introduction

- Data Structures can be defined as a representation of data along with its associated operations. It is the way of storing and organizing data in a computer system so that it can be used efficiently.
- This organization can be in the form of a group of data elements stored under one name. Here, the data elements are referred as members of data structure
- Depending on the type of structures, the members can be of different types and lengths.
- Algorithms are used to manipulate the data structures in a number of different ways, like sorting the data elements or searching a particular data item.
- The design and ~~manipulation~~ implementation of a typical data structure is associated with the definition of operations that can be performed on the data structure. The specification of these data structure operations is done with the help of algorithms.
- Data structures help in storing, organizing and analyzing the data in logical manner.

Characteristics of Data structures

1. It depicts the logical representation of data in computer memory.
2. It represents the logical ~~rep~~ relationship between the various data elements.
3. It helps in efficient manipulation of stored data elements.
4. It allows the programs to process the data in an efficient manner.

* Types of Data structures.



- Data structures are primarily divided into two classes- primitive and non-primitive.
- Primitive Data structures include all the fundamental data structures that can be directly manipulated by machine level instructions. Some common examples are integer, character, real, boolean, etc.

- Non-primitive data structures refers to all the data structures that are derived from one or more primitive data structures. The objective of creating non-primitive data structures is to form sets of homogeneous or heterogeneous data elements. It is further classified as linear and non-linear data structures. In linear data structures, all the data elements are arranged in linear or sequential fashion. Some examples are arrays, stacks, queues, linked lists, etc. In non-linear data structures, there is no definite order or sequence in which data elements are arranged. They are stored in a hierarchical fashion. Some examples are trees and graphs.

Linear Data Structures

↳ 1 Arrays

a. An array is a collection of similar datatypes of data elements stored at consecutive location in the memory.

b. The group of array elements is referred with a common name called array name.

c. In C, array index starts with 0.

d. Syntax of array :

```
datatype array_name [arraysize]={Data elements};
```

e. Examples of array include list of integers, group of names etc.

eg. `int roll_no[72];`

f. Array elements are accessed using the index identifier

g. Logical representation of array.

Array index →	arr[0]	arr[1]	arr[2]	arr[3]	arr[4]
	11	13	15	16	18
Address. →	6000	6001	6002	6003	6004

h. Application : Arrays are particularly used in programs that require storing large collection of similar type data elements.

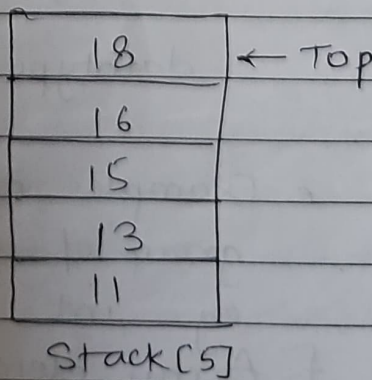
↳ 2. Stacks.

a. Stack is a linear data structure that maintains a list of elements in such a manner that elements can be inserted or deleted only from one end of the list. This end is referred as top of the stack.

b. Stack is based on Last In First Out (LIFO) principle, which means the element that is inserted last is the first element removed from the stack.

c. A stack of book can be considered similar to stack data structures, wherein books cannot be inserted or removed from the middle.

d. Logical Representation of Stack



e. Application: Stacks are typically used in the implementation of system processes, such as compilation and program control.

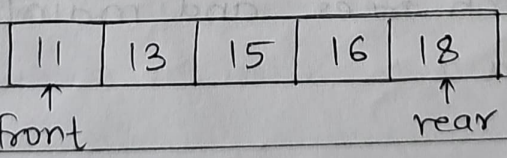
↳3. Queues

a. Queue is a linear data structure that maintains a list of elements in such a manner that elements are inserted from one end of the queue called rear end and deleted from the other end called front.

b. Queue is based on the principle First In First Out (FIFO), which means the elements that is added first to the queue is also the one that is first removed from the queue.

c. A queue of people standing at a bus stop is an example of queue as people join the queue from the back and leave the queue from the front.

d. Logical Representation of Queue



e. Application of Queue:
Queues are typically used in the implementation of key system processes such as CPU scheduling, resource sharing, etc.

PAGE No.
DATE / /

Non-linear data structures

↳ 4. Trees.

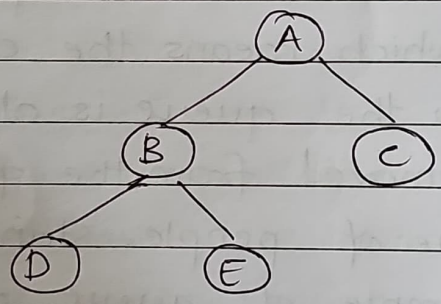
a. Tree is a linked data structure that arranges its nodes in the form of a hierarchical tree structure

b. Each node consists of zero, one or two child nodes.

c. The node present at the top of the tree is called the root node

d. Data is accessed from the tree data structure using various tree transversal methods

e. Representation of tree



f. Application of Trees.

Tree data structure is typically used for storing hierarchical data, implementing search trees and maintaining sorted data.

PAGE No.
DATE / /

5. Graphs.

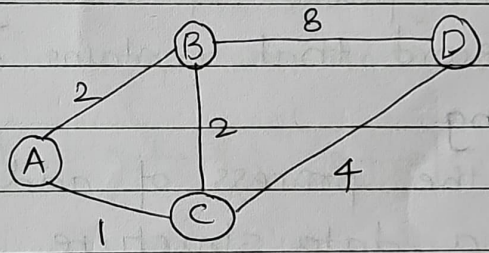
a. Graph is a linked data structure that comprises of a group of vertices called nodes and group of edges.

b. An edge is nothing but a pair of vertices.

c. Graph data structures realizes mathematical concepts of graphs.

d. The edges of a graph are typically associated with certain values called as weights. This helps to compute the cost of transversing the graph through a certain path

e. Representation of Graph.



f. Application of Graphs; One of typical application of graphs is in the modelling of network systems. It helps to compute the cost of transmitting data from a particular network path.

* Data Structure Operations

There are several common operations associated with data structures that are used for manipulating the stored data. While defining a data structure, you also need to define these associated operations. The following operations are most frequently performed on any data structure type:-

↳ 1. Traversing

It is the process of accessing each record of a data structure exactly once.

↳ 2. Searching

It is the process of identifying the location of a record that contains a specific key value

↳ 3. Inserting

It is the process of adding a new record in to a data structure

↳ 4. Deleting

It is the process of removing an existing record from a data structure.

↳ 5. Sorting

It is the process of arranging the records of a data structure in a specific order, such as alphabetical, ascending or descending

↳ 6. Merging

It is the process of combining the records of two different sorted data structures to produce a single sorted data set.

Linear data structures

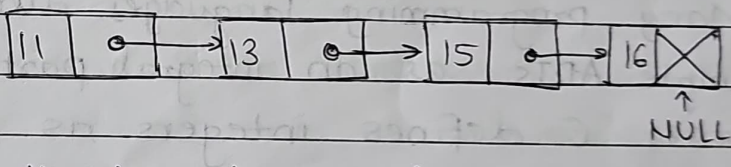
↳ 6. Linked lists.

a. Linked list is a data structure used for storing data in the form of a list.

b. It comprises of multiple nodes connected to each other through pointers. Each node comprises of two parts. One part consists of the data value and the other part consists of the pointer to the next node in the list.

c. Linked list eliminate one of the main disadvantages of arrays, that is, inefficient utilization of memory space.

d. Logical representation of linked lists.



e. Application of linked list:

Linked lists are used in situations where there is a need for dynamic memory allocation.

* Concept of ADT or Abstract Data Types.

To process data with a computer, we need to define the data type and the operation to be performed on the data
The definition of the datatype and the definition of the operation to be applied to the data remains hidden from the user. This is called as Abstract Data Types (ADTs). User of an ADT needs to know the operations available for the datatype but does not need to know how they are applied

Simple ADTs
Many programming languages already define some simple ADTs as an integral part of the language. for. eg. C defines integers as simple ADTs. It also defines the various operations applied on it.
Examples of Simple ADTs - integer, real, character, pointer, etc.
Complex ADTs
Complex ADTs are stack ADTs, queue ADTs and so on. Even though they are useful, they are not implemented or used in most languages. To be efficient, these ADTs should be created and stored in the library of the computer to be used.